

Guía para Desarrolladores — Repos con TDD + IA

Guía para Desarrolladores — Repos con TDD + IA

Audiencia: cualquier desarrollador que va a trabajar en un repo de `kriptos-io` provisionado por la skill `repo-provisioning`. **No necesitás conocer la skill por dentro** — solo cómo desarrollar dentro del repo que ya recibiste.

Si tu trabajo es **mantener o mejorar la skill** que provisiona estos repos, esa documentación está en otra página: [Guía Técnica — Mantenimiento de la Skill `repo-provisioning`](#).

1. Qué deja listo el repo

Cuando recibís un repo provisionado, ya viene con:

```
1 .
2   ├── CLAUDE.md           # Contexto + workflow del agente IA
3   ├── AGENTS.md           # Pointer a CLAUDE.md (para
4   |   Codex/Cursor/Aider)
5   ├── CONTRIBUTING.md     # Guía humana: stack, branching, CI/CD
6   ├── sonar-project.properties # SonarCloud configurado
7   ├── Dockerfile         # Template del lenguaje/servicio
8   ├── .github/
9   |   ├── pull_request_template.md # PR con DoD checklist
10  |   ├── ISSUE_TEMPLATE/{bug,feature}.md
11  |   └── workflows/{ci-cd-dev,ci-cd-prod}.yaml
12  ├── .claude/
13  |   ├── settings.json    # Permisos automáticos + hooks
14  |   └── hooks/           # block-main, block-dangerous, enforce-
15  |   tdd
16  |   ├── rules/           # Reglas cargadas por path-matching
17  |   └── skills/security-review/
18  ├── specs/_template.md   # Plantilla para escribir specs
19  └── docs/architecture/_adr_template.md
```

En GitHub, el repo ya tiene:

- Branch protection en `main` (PR + review aprobado obligatorios).
- Equipos `development-team` y `qa-team` con permisos push.
- Proyecto SonarCloud vinculado vía CI.

2. Responsabilidades de cada documento

Documento	Para qué sirve
-----------	----------------

CLAUDE.md	Lo que el agente IA carga en cada conversación: workflow obligatorio, Definition of Done, Limits of Autonomy. Es fuentes de verdad de comportamiento del agente — corto a propósito.
AGENTS.md	Pointer a CLAUDE.md para otros agentes. No editar.
CONTRIBUTING.md	Para humanos. Stack del lenguaje, branching, qué hace cada pipeline, naming conventions. Lo lee el agente cuando lo necesita.
.claude/rules/*.md	Restricciones duras (NEVERs/MUSTs) por dominio. El agente las carga solo cuando edita archivos que matchean los paths: del frontmatter.
.claude/hooks/*.sh	Enforcement local automático. block-main-branch y block-dangerous-commands bloquean. enforce-tdd solo avisa (no rompe brainstorming).
.claude/settings.json	Permisos pre-aprobados para comandos seguros + registro de hooks. Reduce prompts.
.github/pull_request_template.md	Te fuerza a linkear spec + ticket y a marcar el checklist de DoD al abrir un PR.
.github/ISSUE_TEMPLATE/	Plantillas para bugs / features. Una feature aceptada se promueve a spec antes de implementar.
specs/<feature>.md	Contrato de qué hace la feature, AC testables, edge cases. Sin spec

	aprobada no hay implementación.
docs/architecture/<name>- adr.md	Solo si hay una decisión de aplicación con tradeoffs reales (DynamoDB vs RDS, push vs pull). Las decisiones de infra viven en el monorepo de Terraform. Es opcional , no parte de la DoD.

3. Primera vez que accedés al repo

3.1. Clonar y verificar entorno

```
1 gh repo clone kriptos-io/<repo-name>
2 cd <repo-name>
```

Necesitás:

- gh (GitHub CLI) autenticado.
- Claude Code (CLI o plugin de IDE).
- Plugin `superpowers` instalado:

```
1 | claude plugin install superpowers@claude-plugins-official
```

- Stack del lenguaje del repo (ver `CONTRIBUTING.md`).

3.2. Leer en este orden

1. `CLAUDE.md` — workflow, DoD, límites del agente.
2. `CONTRIBUTING.md` — stack del repo y pipelines.
3. `.claude/rules/tdd.md` — disciplina RED→GREEN→REFACTOR.
4. `specs/_template.md` — la estructura que vas a usar.

3.3. Configurar variables del repo

Si el repo es **EMR Serverless** o **Lambda**, configurar en `Settings → Secrets and variables → Actions → Variables`:

- `EMR_SERVERLESS_APPLICATION_ID_DEV` / `_PROD`
- `S3_BUCKET_NAME_DEV` / `_PROD`
- Equivalentes según stack (ver `CONTRIBUTING.md`).

Son **Variables**, no **Secrets** — los IDs de recursos no son sensibles.

3.4. Validar el CI

Hacé un commit dummy en un branch de prueba y verificá que el pipeline de dev corre limpio. Confirma que SonarCloud, Snyk y workflows están bien conectados antes de empezar feature work real.

4. Flujo por cada feature

Toda feature pasa por estos 9 pasos, sin excepciones. Notar la división de responsabilidades: el **humano** trae la idea y aprueba la spec; el **agente** refina, planifica y ejecuta.

Paso	Quién lo hace	Qué hace
1. Branch	Humano	Crea branch desde Jira
2. Draft de spec	Humano	Rellena specs/<feature>.m d con la idea inicial
3. Brainstorming	Humano + agente	Refina la spec — el agente desafía, el humano decide
4. Aprobar spec	Humano	Commitea y opcionalmente PR de la spec sola
5. Plan	Agente	writing-plans desde la spec aprobada
6. TDD	Agente	executing-plans con ciclo RED→GREEN→REFAC TOR
7. Verificar	Agente	verification- before- completion
8. Security review	Agente	/security-review cuando aplica

9. PR	Humano + agente	requesting-code-review y gh pr create
-------	-----------------	---------------------------------------

El agente nunca decide qué hace la feature. Solo refina lo que el humano trae y lo materializa en código una vez aprobado.

4.1. Crear branch desde Jira (humano)

```
1 git checkout main
2 git pull
3 git checkout -b KT-XXXX-short-description-in-english
```

Branch names en **inglés** siempre.

4.2. Draft de spec (humano)

Copía `specs/_template.md` a `specs/<feature-name>.md` y rellena lo que tenés claro:

- **Problem:** qué problema resuelve, para quién.
- **Goals:** qué debe hacer la feature (testable).
- **Comportamiento esperado:** happy path + edge cases que ya identificaste.
- **Open questions:** dudas explícitas para resolver con el equipo o con brainstorming.

No tiene que estar perfecto. Es la semilla. AC vagos y huecos están permitidos en este punto. El brainstorming los va a resolver.

El agente no genera la idea. La idea viene del humano (PM, tech lead, dev). El agente refina lo que el humano trae.

4.3. Brainstorming sobre el draft (humano + agente)

Con el draft commiteado en disco, abrir Claude Code y correr:

```
1 /superpowers:brainstorming
```

El agente **lee la spec draft** y la **desafía**:

- "Mencionás X pero no especificás cuándo se invalida."
- "Este AC no es testable — ¿cómo lo medimos?"
- "Falta edge case: input vacío, timeout, doble submit."
- "Asumís Y — ¿está confirmado?"

El humano responde, decide, y **edita la spec** según las respuestas. Brainstorming **NO escribe la spec** — la refina con la conversación.

Iterar hasta que:

- AC son testables (cada uno mapea a al menos un test).
- Edge cases identificados.
- Open questions resueltas o explícitamente diferidas.
- Out-of-scope marcado.

4.4. Aprobar la spec (humano)

Commitear la spec final:

```
1 | chore: spec for <feature> (KT-XXXXX)
```

Idealmente abrir un PR de la spec sola si el equipo quiere revisar. **Mergear la spec antes del código.**

A partir de acá, **el contrato existe**. El código que venga después tiene un blanco al cual apuntar.

4.5. Plan de implementación (agente)

```
1 | /superpowers:writing-plans
```

Genera un plan con tareas atómicas. **Cada tarea debe tener un paso de test antes del paso de código.** Si no, está mal estructurada.

4.6. Ejecución TDD (agente)

```
1 | /superpowers:executing-plans
```

Para tareas paralelizables (sin dependencias entre sí):

```
1 | /superpowers:subagent-driven-development
```

Ciclo por cada tarea:

1. **RED**: test que falla, commit:

```
1 | chore: <behavior> (failing)
```

2. **GREEN**: implementación mínima que hace pasar el test:

```
1 | feat: <behavior> (passing)
```

3. **REFACTOR** (si hay cleanup):

```
1 | refactor: <behavior>
```

Si después de 3 intentos no pasa, **reportar BLOCKED**. No improvisar.

4.7. Verificación previa a declarar done (agente)

```
1 | /superpowers:verification-before-completion
```

Corre los comandos reales y lee la salida real. **"Tests pasan en mi máquina" no es done — CI verde es done.**

4.8. Security review cuando aplica (agente)

Obligatorio si tocás:

- Autenticación o autorización.
- Manejo de secretos.
- API pública.
- Nuevas entidades de dominio.

```
1 | /security-review
```

Genera el threat model en `docs/security/<feature>-threat-model.md`.

4.9. Code review interno y PR (humano + agente)

```
1 | /superpowers:requesting-code-review
```

Después abrir el PR:

```
1 | gh pr create
```

El template del PR fuerza:

- Link a la spec.
- Link al ticket Jira.
- Checklist de Definition of Done.

CI debe estar verde antes de pedir review humano. Sin verde, el reviewer no debe abrirlo.

5. Definition of Done

Una feature **NO** está done hasta que **todos** estos puntos sean verdaderos:

- ☐ Spec aprobada y commiteada en `specs/<feature>.md`.
- ☐ Test fallido committeado primero como `chore: <behavior> (failing)`.
- ☐ Implementación committeada como `feat: <behavior> (passing)`.
- ☐ Refactor committeado si hubo cleanup: `refactor: <behavior>`.
- ☐ Coverage $\geq 80\%$ (SonarCloud quality gate verde).
- ☐ Snyk + SonarCloud verdes en CI.

- ☐ `/security-review` ejecutado si aplica.
 - ☐ PR abierto con link a spec y ticket.
 - ☐ CI verde **antes** de solicitar review humano.
-

6. Limits of Autonomy del agente

El agente IA **NO** debe hacer ninguna de estas acciones sin tu aprobación explícita:

- Push directo a `main`.
- Merge de un PR.
- Disparar workflows `deploy/*` o `release/*`.
- Instalar dependencia nueva sin justificación.
- Regenerar lockfile mid-PR.
- Modificar workflows de CI.
- Modificar branch protection o settings del repo.
- Deshabilitar hook, CI check o scanner para que un build pase.
- Usar `--no-verify`, `--force`, `--force-with-lease`.
- Editar un test para hacerlo pasar (debe corregir la implementación).
- **Decidir el alcance o comportamiento de una feature sin spec aprobada.**

Si el agente intenta una de estas → frenalo y pedí explicación.

7. Convención de commits y branch

1	<code>chore: spec for <feature> (KT-XXXXX)</code>	← spec aprobada
2	<code>chore: <behavior> (failing)</code>	← fase RED
3	<code>feat: <behavior> (passing)</code>	← fase GREEN
4	<code>refactor: <behavior></code>	← fase REFACTOR
5	<code>deploy: ...</code>	← reservado para CI/CD
6	<code>release: ...</code>	← reservado para CI/CD

- Mensajes en **inglés** siempre.
 - Branch: `KT-XXXXX-short-description` en **inglés**.
-

8. Protocolo BLOCKED

El agente debe parar y reportar `BLOCKED: <motivo>` cuando:

- La spec es ambigua o contradictoria.
- Un test falla después de 3 intentos.
- Un test existente parece incorrecto (NO reescribirlo).

- Se detecta un secreto hardcodeado.
- La verificación produce output inesperado que no puede explicar.

NO improvisar. Esperar input humano.

9. Troubleshooting común

Síntoma	Causa probable	Acción
Push a main bloqueado	Hook <code>block-main-branch</code> activo	Crear feature branch
Coverage rojo en SonarCloud	Tests faltantes o exclusiones mal	Revisar <code>sonar-project.properties</code> y agregar tests
Pipeline prod no avanza	Falta QA approval	Pedir aprobación en el PR al equipo de QA
<code>enforce-tdd</code> advierte sin razón	Hook es advisory, no rompe nada	Ignorar; el gate duro está en CI
Variables de EMR no resuelven	No configuradas en GitHub Variables	Settings → Secrets and variables → Actions → Variables
Brainstorming intenta escribir código	Mal uso del comando	Recordá que brainstorming solo refina la spec . Hasta que la spec esté aprobada, no hay código.

Última actualización: 2026-05-08. Si la skill cambia el comportamiento del repo provisionado, esta página se actualiza acá.